

SAFE-AGENT:

A Conceptual Framework for Secure Multi-Agent RAG Systems with Memory, Hallucination Detection, and Risk-Aware Decision Making

By -Animey Chaturvedi

B.Tech Computer Science & Engineering
JECRC University, Jaipur, India

Email: animeychaturvedi@gmail.com

GitHub: github.com/Animey13

LinkedIn: linkedin.com/in/animey-chaturvedi-0b494a288/

Conceptual Framework Paper

Prepared for Submission to Undergraduate Research Symposia and Student Conferences

June 2026

Keywords:

Multi-Agent Systems · Retrieval-Augmented Generation · Hallucination Detection · Memory-Augmented Agents · Agent Security · Risk Assessment · LLM Safety

Scope Notice: This paper presents a conceptual framework and architectural proposal for a secure multi-agent system. It does not report implemented experimental results. All evaluation approaches described herein are proposed methodologies for future empirical validation.

Abstract

As Large Language Model (LLM)-based autonomous agents are deployed in increasingly consequential environments, the absence of integrated safety mechanisms presents a growing risk. Current systems address individual concerns — factual grounding, memory, or access control — in isolation, with no unified architecture that coordinates these protections. This paper proposes **SAFE-AGENT**, a conceptual framework for a secure multi-agent system designed to address this gap through four coordinated components: (1) a **Secure Retrieval-Augmented Generation (RAG) module** intended to improve factual grounding while resisting adversarial document injection; (2) a **Memory-Augmented Context Engine (MACE)** designed to maintain episodic and semantic context with cryptographic integrity guarantees; (3) a **Hallucination Detection and Correction Module (HDCM)** that proposes a cross-agent consensus mechanism to flag and correct uncertain claims before they propagate; and (4) a **Risk-Aware Decision Engine (RADE)** that evaluates agent action sequences against dynamic risk policies before execution. We present the theoretical design of each component, analyze the threat models each is intended to address, and outline a proposed evaluation methodology against standard benchmarks. SAFE-AGENT is offered as a conceptual foundation and research roadmap; empirical validation is identified as essential future work.

1. Introduction

The emergence of autonomous agents built on Large Language Models (LLMs) has expanded the scope of AI-assisted decision-making into domains with significant real-world consequences — from clinical research synthesis to security operations and financial analysis. As these systems grow more capable, so do the consequences of their failure modes. Three interconnected vulnerabilities are of particular concern: **hallucination** — the generation of plausible but factually incorrect content; **context degradation** — the loss of coherent memory and task state across long agent interactions; and **adversarial manipulation** — the exploitation of agent reasoning through prompt injection, data poisoning, or capability abuse.

While meaningful work addresses each of these challenges individually — Retrieval-Augmented Generation [Lewis et al., 2020] for factual grounding, memory architectures [Park et al., 2023] for context persistence, and adversarial robustness research [Greshake et al., 2023] for prompt injection — no existing framework combines them into a unified, security-first multi-agent architecture. The result is that deploying a capable multi-agent system currently requires assembling disparate safety components with no formal guarantees about their interactions or their collective coverage.

This paper introduces **SAFE-AGENT**, a conceptual framework designed to fill this gap. We propose an architecture in which every reasoning, retrieval, and action step passes through complementary

safety mechanisms that are designed to compensate for each other's failure modes. Our contributions are as follows:

- **Unified framework:** We propose SAFE-AGENT as the first conceptual architecture integrating multi-agent orchestration, secure RAG, episodic memory, hallucination detection, and risk-aware planning into a single coherent design.
- **MACE:** We design a Memory-Augmented Context Engine with a two-phase write-commit protocol and cryptographic integrity verification intended to prevent false information from being stored and later retrieved as ground truth.
- **HDCM:** We propose a Hallucination Detection and Correction Module based on cross-agent consensus and evidence grounding, designed to flag uncertain claims before they propagate to dependent agents or external actions.
- **RADE:** We introduce a Risk-Aware Decision Engine that evaluates proposed action sequences against a scored risk taxonomy before execution, with automatic rollback capabilities for reversible transactions.
- **Evaluation roadmap:** We propose concrete benchmark suites and evaluation protocols for future empirical validation of the framework.

Paper Scope: SAFE-AGENT is a proposed architecture, not a deployed or benchmarked system. The framework contributions are conceptual and design-oriented. Section 5 describes a proposed evaluation methodology that constitutes planned future work. Claims throughout this paper are framed as design intent, theoretical analysis, or architectural proposal.

2. Related Work

2.1 Multi-Agent LLM Systems

The multi-agent paradigm for LLMs emerged from the recognition that decomposing complex tasks among specialized agents can improve overall performance [Wu et al., 2023; Hong et al., 2023]. AutoGen [Wu et al., 2023] demonstrated that conversational agent networks can address software engineering tasks that exceed single-model capability. CAMEL [Li et al., 2023] explored role-playing agents for collaborative problem solving. MetaGPT [Hong et al., 2023] formalized software development as multi-agent workflow. These systems demonstrate strong task performance; however, they do not incorporate integrated safety guarantees — a compromised or hallucinating agent can propagate errors across an entire pipeline without detection.

2.2 Retrieval-Augmented Generation

RAG [Lewis et al., 2020] improves factual grounding by conditioning LLM generation on retrieved documents. Subsequent work improved multi-hop retrieval [Trivedi et al., 2022] and self-reflective retrieval strategies [Asai et al., 2023]. REALM [Guu et al., 2020] integrated retrieval into pretraining. A critical gap in existing RAG frameworks is the assumption that the retrieval index is trustworthy; none of the foundational RAG architectures incorporates mechanisms to detect adversarial

document injection or verify index integrity — a vulnerability SAFE-AGENT's Secure RAG module is designed to address.

2.3 Hallucination Detection

Hallucination remains a significant challenge for deployed LLMs. SelfCheckGPT [Manakul et al., 2023] samples multiple generations and measures inconsistency as a proxy for hallucination likelihood. FActScoring [Min et al., 2023] decomposes responses into atomic claims and retrieves verification evidence per claim. Chain-of-Verification [Dhuliawala et al., 2023] generates self-verification questions from model outputs. These approaches are effective on isolated statements but face scalability challenges in multi-agent pipelines where a single claim may be cited by multiple downstream agents. HDCM's proposed *tiered verification protocol* is designed to address this by applying heavy verification selectively, based on confidence signals from the generating agent.

2.4 Memory in Autonomous Agents

Generative Agents [Park et al., 2023] introduced memory retrieval and reflection in simulated social environments, demonstrating that memory meaningfully improves long-horizon task coherence. MemoryBank [Zhong et al., 2023] applied Ebbinghaus forgetting curve models to agent memory weighting. Voyager [Wang et al., 2023] maintained a procedural skill library as memory for open-ended exploration. These systems treat memory as a capability enhancement; MACE extends this by treating memory as a security boundary — applying integrity verification to prevent false information from contaminating the shared memory pool.

2.5 Agent Security

Prompt injection attacks [Perez & Ribeiro, 2022; Greshake et al., 2023] have demonstrated that malicious instructions embedded in retrieved documents can hijack agent behavior. Indirect prompt injection [Greshake et al., 2023] is particularly relevant in RAG settings, where retrieved content is often implicitly trusted. Constitutional AI [Bai et al., 2022] and RLHF [Ouyang et al., 2022] improve model-level alignment but do not address runtime adversarial inputs that arrive after deployment. SAFE-AGENT builds on this literature by proposing runtime architectural defenses that operate independently of model weights.

3. SAFE-AGENT: Proposed Architecture

SAFE-AGENT is designed around a four-layer defense stack, each layer providing complementary guarantees while communicating through a secure inter-agent message bus. The architecture comprises five primary components: the Orchestrator, MACE, HDCM, RADE, and the Secure RAG Module.

Table 1: SAFE-AGENT Component Overview and Design Intent

Component	Proposed Role	Intended Safety Guarantee
Orchestrator	Task decomposition, agent coordination, state management	Sandboxed execution, audit logging
Secure RAG Module	Dual-index retrieval with adversarial content filtering	Index integrity assurance, provenance tracking
MACE	Episodic + semantic memory with integrity verification	Context integrity, tamper detection
HDCM	Cross-agent consensus, confidence scoring, correction	Reduced hallucination propagation
RADE	Risk-scored action planning, policy enforcement	Action safety, rollback capability

3.1 Orchestrator

The Orchestrator is the central coordination component of SAFE-AGENT. It would receive high-level task specifications, decompose them into subtasks using hierarchical planning, and assign subtasks to specialized agents drawn from a typed agent pool. The Orchestrator is intended to maintain a **global state graph** — a directed acyclic graph (DAG) of task nodes, their dependencies, execution states, and associated trust scores. Each task node in the DAG is designed to carry a **risk budget** derived from RADE that constrains the set of permitted actions for its assigned agent.

All inter-agent communication is intended to route through a **Secure Message Bus (SMB)** designed to enforce message authentication (HMAC-SHA256), rate limiting, and content filtering. The Orchestrator is designed to apply a **minimum-privilege routing policy**: agents would receive only the context fragments relevant to their assigned subtask, limiting lateral information access. Agent outputs are designed to pass through HDCM before being committed to MACE or forwarded to dependent agents.

3.2 Secure RAG Module

The Secure RAG Module is designed to extend standard dense retrieval with three additional security layers: index integrity verification, adversarial document filtering, and provenance tracking.

3.2.1 Dual-Index Retrieval

The proposed architecture maintains two separate vector indices: a **primary corpus index** of curated, high-trust documents, and a **live retrieval index** updated with externally retrieved content. Queries would execute against both indices simultaneously. Documents from the primary index are assigned high trust scores; documents retrieved solely from the live index would be marked as *unverified* and subjected to additional HDCM verification before use in generation. Documents in the primary index are intended to be signed with publisher certificates and their hashes stored in a Merkle tree, such that tampering would trigger automatic quarantine.

3.2.2 Adversarial Content Filtering

Prior to entering the generation context, retrieved documents would pass through an **Adversarial Content Scanner (ACS)**, a proposed classifier designed to detect prompt injection patterns, jailbreak templates, and instruction override attempts. The design envisions training on established prompt injection datasets such as those from Greshake et al. [2023] augmented through structured red-teaming. Documents exceeding a configurable injection risk threshold would be quarantined and flagged for human review, while the requesting agent receives a sanitized notice that the document was unavailable.

3.2.3 Provenance Chain

SAFE-AGENT is designed such that every claim in a generated response is traceable to its source document through a provenance graph maintained in MACE. This would enable post-hoc auditing: when a hallucination or error is later detected, the provenance chain is intended to identify the responsible source, enabling targeted remediation rather than full pipeline re-evaluation.

3.3 MACE: Memory-Augmented Context Engine

MACE proposes two complementary memory stores inspired by the Atkinson-Shiffrin model of human memory: an **Episodic Memory Store (EMS)** for interaction history and a **Semantic Memory Store (SMS)** for distilled knowledge.

3.3.1 Episodic Memory Store

The EMS is designed to record agent interactions as structured episode objects containing task context, agent actions, retrieved documents, generated outputs, HDCM confidence scores, and RADE risk assessments. Episodes are intended to be stored with **HMAC-SHA256 signatures** over episode content; before retrieval, signatures would be verified, and tampered episodes rejected and quarantined. The EMS is proposed to implement a **temporal decay function** based on Ebbinghaus forgetting curves to weight recent episodes more heavily during retrieval, while preserving high-importance historical episodes regardless of age.

3.3.2 Semantic Memory Store

The SMS is designed to store distilled knowledge as **knowledge graph triplets** (entity, relation, entity) extracted from verified agent interactions. Entities would be embedded using contrastive sentence encoders [Gao et al., 2021], enabling fuzzy semantic retrieval. Multi-hop reasoning would be supported by chaining up to three triplet hops, assembling a rich contextual neighborhood to improve reasoning depth without expanding prompt length.

3.3.3 Memory Integrity Protocol

A key design feature of MACE is the **two-phase write-commit protocol**: a writing agent proposes a memory update; HDCM verifies that the proposed content is consistent with existing verified knowledge; only upon HDCM approval does MACE commit the write. This protocol is designed to prevent a hallucinating or compromised agent from introducing false information into the shared

memory pool, which would otherwise be retrieved and treated as verified fact by subsequent agents.

3.4 HDCM: Hallucination Detection and Correction Module

HDCM is the epistemic verification layer of SAFE-AGENT. It is designed to operate through a three-stage pipeline: claim extraction, cross-agent consensus, and evidence-based correction.

3.4.1 Claim Extraction

Agent outputs would be decomposed into **atomic claims** using a fine-tuned claim extraction model [cf. Raffel et al., 2020]. Each claim is intended to be a self-contained, verifiable proposition. Claims would be classified into three types: *factual claims* (verifiable against external sources), *logical claims* (verifiable by formal reasoning), and *referential claims* (verifiable against retrieved documents). Each type would undergo a distinct verification pathway optimized for its verification mechanism.

3.4.2 Cross-Agent Consensus

For high-stakes claims, HDCM is designed to invoke a **Consensus Panel**: two additional specialized agents — a Verifier Agent and a Devil's Advocate Agent — independently assess the claim. The Verifier searches for supporting evidence; the Devil's Advocate attempts to falsify the claim. A **confidence score** $C \in [0, 1]$ would be computed as a weighted combination of panel votes and retrieval evidence strength. Claims with C below a configurable *uncertainty threshold* would be flagged as uncertain; claims with C below a *hallucination threshold* would trigger the correction subroutine.

3.4.3 Correction and Substitution

When a claim is flagged as likely hallucinated, HDCM is designed to attempt correction before falling back to explicit abstention. The correction subroutine would re-query the Secure RAG module with targeted retrieval prompts derived from the flagged claim, regenerate the claim under a lower-temperature decoding regime, and re-evaluate the corrected claim against the same thresholds. If the corrected claim meets the confidence threshold, it would substitute the original. Otherwise, the claim would be replaced with an explicit acknowledgment of uncertainty, ensuring SAFE-AGENT never silently propagates unverified information.

3.5 RADE: Risk-Aware Decision Engine

RADE is designed to govern agent action selection by evaluating proposed actions against a dynamic risk model before execution, drawing on principles from formal verification in safety-critical software engineering.

3.5.1 Risk Taxonomy

RADE is proposed to maintain a **Risk Taxonomy** of action categories organized in a three-level hierarchy: domain (e.g., Network, Filesystem, External API), operation type (e.g., Read, Write,

Execute, Delete), and scope (e.g., Local, Remote, Persistent). Each category would carry a **base risk score** calibrated against established risk frameworks such as MITRE ATT&CK tactic severity ratings. The effective risk score for a proposed action would be computed as:

$$r_{\text{eff}} = r_b \times \alpha_{\text{context}} \times \beta_{\text{agent}} \times \gamma_{\text{history}}$$

where α_{context} scales risk based on task sensitivity, β_{agent} reflects the trust score of the proposing agent, and γ_{history} applies a recency-weighted penalty for agents with a history of risky or failed actions. Actions would be **approved** if r_{eff} falls below a task-level risk threshold, **escalated** to human review within an intermediate range, and **blocked** above a defined ceiling. Importantly, RADE is designed to evaluate *action sequences*, not only individual actions, using a cumulative risk integral to detect decomposition attacks in which multiple low-risk steps collectively achieve a high-risk outcome.

3.5.2 Rollback and Recovery

RADE is designed to implement a transactional action model: all agent actions that modify external state would be wrapped in reversible transactions with checkpoints. If a risk violation is detected mid-execution, RADE would trigger automatic rollback to the last safe checkpoint. The rollback log would be preserved in MACE for post-incident analysis, creating an auditable trail of both successful and failed agent actions.

4. Theoretical Security Analysis

We analyze SAFE-AGENT against three representative threat models to reason about the coverage and limitations of the proposed design. This analysis is theoretical; empirical security evaluation is identified in Section 5 as essential future work.

4.1 Threat Model T1: Prompt Injection via Retrieved Documents

Threat description: An adversary embeds malicious instructions in documents that are indexed or retrieved by the Secure RAG module, intending to override agent instructions or exfiltrate sensitive information.

Proposed mitigation: The ACS classifier is designed to detect this class of attack by filtering retrieved documents before they enter the agent context. Additionally, all retrieved content is proposed to be presented in a constrained context role with explicit system-level framing that demotes its instruction authority. The Dual-Index design further reduces exposure by limiting high-authority context to documents from the curated primary index. A meaningful gap in coverage is that the ACS classifier's effectiveness against novel or obfuscated injection patterns is unknown without empirical evaluation.

4.2 Threat Model T2: Memory Poisoning

Threat description: A hallucinating or compromised agent attempts to write false information to MACE, which subsequent agents then retrieve and treat as verified fact.

Proposed mitigation: The two-phase write-commit protocol requires HDCM approval for all memory writes. In theory, a false claim can only be committed if it achieves a confidence score above the verification threshold, which requires corroborating retrieval evidence and consensus panel agreement. A limitation of this design is that the false positive rate of HDCM — the rate at which false claims incorrectly pass verification — is a critical parameter that can only be determined through empirical testing on representative benchmark datasets.

4.3 Threat Model T3: Privilege Escalation via Action Decomposition

Threat description: An agent attempts to execute high-risk actions by decomposing a prohibited operation into a sequence of individually low-risk steps that collectively achieve the same effect, bypassing per-action risk checks.

Proposed mitigation: RADE's cumulative risk integral evaluates action sequences, not only individual actions, and is designed to detect this decomposition pattern. Agent trust scores that decay after action rejection are intended to raise the effective risk score for potentially adversarial agents over time. A theoretical limitation is that sufficiently long, low-risk decomposition sequences may remain below the cumulative risk threshold; calibration of the integral parameters is an open research question requiring empirical study.

Analytical Caveat: The above threat analyses are reasoning-based assessments of the proposed design. They do not constitute formal security proofs, and the actual effectiveness of each mitigation depends on implementation choices, parameter calibration, and the specific threat environment. Peer review and adversarial red-teaming of a working implementation would be required before any security claims can be made quantitatively.

5. Proposed Evaluation Methodology

This section describes how SAFE-AGENT *would be* evaluated once a working implementation is developed. We specify the benchmarks, baselines, and metrics that a future empirical study should employ. This serves both as a research roadmap and as a transparency mechanism: reviewers and readers can assess whether the proposed evaluation design would be sufficient to validate the framework's claims.

5.1 Proposed Benchmarks

- **AgentBench [Liu et al., 2023]:** A standard evaluation suite for LLM agent capabilities across eight task categories including web navigation, database interaction, and code execution. This benchmark would assess whether the safety mechanisms of SAFE-AGENT impose an unacceptable task performance penalty.
-

- **HaluEval [Li et al., 2023]:** A hallucination benchmark across dialogue, QA, and summarization tasks. This would be the primary benchmark for measuring HDCM's effectiveness at reducing hallucination rates relative to unprotected baselines.
- **MITRE ATT&CK Evaluation Alignment:** RADE's risk taxonomy would be cross-referenced against MITRE ATT&CK tactic and technique coverage to assess whether the proposed risk categories align with recognized threat classifications.
- **SafeAgentBench (proposed):** We propose the construction of a purpose-built benchmark for safety-critical agent tasks. This dataset would need to be constructed through a combination of (a) red-teaming sessions with human adversaries, (b) automated adversarial prompt generation, and (c) expert annotation of ground-truth safety labels. Dataset size, collection procedure, and inter-annotator agreement protocols would need to be defined and documented before the benchmark could be used in published evaluation. This benchmark does not currently exist; its design is a contribution of this framework paper.

5.2 Proposed Baselines

A rigorous empirical evaluation would compare SAFE-AGENT against the following reference systems, chosen for their relevance to different aspects of the framework:

- **GPT-4o + Standard RAG:** Representing the current state of practice for grounded LLM agents, without additional safety mechanisms.
- **AutoGen [Wu et al., 2023]:** A well-established multi-agent framework against which overall task performance and coordination overhead can be compared.
- **SelfCheckGPT [Manakul et al., 2023]:** The most directly relevant hallucination detection baseline, against which HDCM's accuracy-latency trade-off should be measured.
- **Security-only baseline:** A system with only the Adversarial Content Scanner applied but no HDCM or RADE, to isolate the contribution of retrieval security independently of reasoning security.

5.3 Metrics

Table 2: Proposed Evaluation Metrics by Component

Component	Primary Metric	Secondary Metric
HDCM	Hallucination rate (% false claims passing verification)	Precision / recall of hallucination detection
Secure RAG / ACS	Injection block rate (% adversarial docs detected)	False positive rate (legitimate docs flagged)
MACE Integrity	Memory poisoning prevention rate	Retrieval latency overhead vs. baseline
RADE	High-risk action block rate	False escalation rate (safe actions blocked)
Full System	End-to-end task completion rate	Average latency and cost per task

5.4 Ablation Design

To quantify the contribution of each individual component, a structured ablation study would progressively remove each SAFE-AGENT component and measure the resulting change in hallucination rate, attack block rate, and task completion. Specifically, the study would compare:

- Full SAFE-AGENT (all components active)
- SAFE-AGENT without RADE
- SAFE-AGENT without HDCM Consensus Panel
- SAFE-AGENT without full HDCM
- SAFE-AGENT without Memory Integrity Protocol
- SAFE-AGENT without Adversarial Content Scanner
- SAFE-AGENT without Dual-Index RAG
- Baseline (no safety components)

This ablation design would allow attribution of performance differences to specific architectural choices rather than the framework as a whole.

6. Illustrative Application Scenarios

To ground the abstract architectural design in concrete application contexts, we describe two scenarios in which the SAFE-AGENT framework is designed to provide meaningful safety improvements over unprotected baselines. These scenarios are *illustrative* — they are intended to make the design motivations tangible, not to report empirical results.

6.1 Scenario: Medical Literature Synthesis

Consider an agent tasked with synthesizing evidence on a clinical intervention from a corpus of peer-reviewed abstracts. In this domain, hallucination is a serious concern: fabricated efficacy statistics or misattributed side-effect profiles could inform incorrect clinical guidance if the agent output is not independently verified.

In this scenario, SAFE-AGENT's HDCM is designed to provide two specific benefits: (1) the evidence grounding step would cross-reference all citation claims against the retrieved corpus, preventing the agent from fabricating supporting references; and (2) claims for which retrieved documents present contradictory evidence would be flagged as uncertain rather than resolved by hallucination. MACE's provenance chain would allow every quantitative claim in a synthesized report to be traced to its source abstract, enabling human reviewers to verify the synthesis. The two-phase memory write protocol would prevent partially-synthesized and unverified claims from being treated as verified background knowledge in subsequent synthesis steps. *Note: whether SAFE-AGENT would succeed in this scenario to a clinically meaningful degree depends on empirical evaluation that has not yet been conducted.*

6.2 Scenario: Security Operations Automation

Consider an agent tasked with analyzing network traffic logs, correlating threat indicators from an external threat intelligence feed, and recommending defensive actions. Threat intelligence feeds represent a particularly attractive attack surface: an adversary who can inject malicious content into the feed can potentially manipulate agent recommendations.

In this scenario, the Secure RAG module's Adversarial Content Scanner is designed to detect prompt injection payloads embedded in threat intelligence documents before they reach the agent context. RADE's action evaluation is designed to require human review for any recommended action above a defined risk threshold — preventing an adversarially manipulated agent recommendation from automatically triggering a defensive action. MACE's episodic memory would log both the original threat intelligence and the agent's reasoning, providing an audit trail for post-incident review. *The effectiveness of these protections in practice — for example, the fraction of real-world threat intelligence injection attacks that the ACS would detect — is an open empirical question.*

7. Limitations and Open Questions

We identify the following honest limitations of the current framework and open questions for future investigation.

7.1 Framework Limitations

- **No implementation exists:** SAFE-AGENT is a conceptual design. All architectural descriptions are proposals rather than implemented systems. The feasibility of certain design choices — such as the two-phase memory commit at low latency — has not been empirically verified.
 - **No benchmark data:** The proposed benchmarks (SafeAgentBench) do not currently exist. Their construction, including data collection, annotation, and quality control, would require significant additional effort and is subject to the same peer-review scrutiny as any published dataset.
 - **Parameter sensitivity:** HDCM's confidence thresholds, RADE's risk scores, and MACE's temporal decay parameters are design choices whose optimal values depend on the deployment domain. Calibration of these parameters would require domain-specific empirical study.
 - **Multi-modal threats not addressed:** SAFE-AGENT is designed for text-based agent pipelines. Adversarial inputs embedded in images, audio, or code artifacts — increasingly relevant for code-executing agents — are not addressed by this framework.
 - **Consensus Panel cost:** Invoking two additional LLM inference calls per high-stakes claim incurs significant latency and cost overhead whose practical impact at deployment scale has not been analyzed.
-

7.2 Open Research Questions

- What are the optimal ACS training distributions for coverage of novel injection techniques not seen during training?
- What HDCM confidence threshold values minimize the sum of hallucination propagation rate and false correction rate across different task domains?
- Can the cumulative risk integral in RADE be made computationally tractable for very long action sequences without sacrificing coverage of decomposition attacks?
- What inter-annotator agreement levels can be achieved for safety labeling in the proposed SafeAgentBench benchmark construction?
- How does SAFE-AGENT's design degrade gracefully when individual components (HDCM, RADE) encounter inputs significantly outside their design distribution?

8. Conclusion

This paper has presented **SAFE-AGENT**, a conceptual framework for a secure multi-agent architecture that integrates retrieval-augmented generation, episodic and semantic memory, hallucination detection, and risk-aware decision-making into a unified design. The core insight motivating the architecture is that safety in agentic AI cannot be achieved through any single mechanism: a hallucination detector without memory integrity allows false information to accumulate across sessions; a secure retrieval module without action-level risk assessment leaves the agent vulnerable to manipulation through legitimate but adversarially-framed information; and memory integrity without retrieval security allows the index that feeds memory to be poisoned at its source.

Each component in the proposed architecture is designed to compensate for the failure modes of the others, creating a defense-in-depth posture that is intended to be more robust than any individual mechanism. We have analyzed the design against three representative threat models and identified the theoretical coverage and gaps of each proposed mitigation.

We are explicit that SAFE-AGENT currently exists as a framework proposal, not a validated system. The next stage of this research is implementation and empirical evaluation against the benchmark suites and baselines described in Section 5. We hope that this framework paper serves as a useful foundation for that work — providing a clear architectural target, an honest threat model analysis, and a concrete evaluation roadmap — and contributes to the growing body of work on trustworthy, safety-critical multi-agent AI systems.

Intended Venues: This paper is appropriate for submission to undergraduate research symposia, student conference tracks at AI/ML venues (e.g., AAAI student program, student workshop at NeurIPS or ICML), and college-level research journals. As a conceptual framework paper, it does not claim experimental results and should not be reviewed against that standard. A working implementation with empirical evaluation would be required for submission to full research conference tracks.

Acknowledgments

The author thanks course instructors, peer reviewers, and any colleagues who provided feedback on earlier drafts of this framework. Future acknowledgments will include collaborators on the implementation phase of this research.

References

- [1] Asai, A., Wu, Z., Wang, B., Sil, A., & Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. arXiv:2310.11511.
 - [2] Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073.
 - [3] Chase, H. (2022). LangChain. GitHub repository. <https://github.com/hwchase17/langchain>.
 - [4] Dhuliawala, S., Komeili, M., Xu, J., Raileanu, R., Li, X., Celikyilmaz, A., & Weston, J. (2023). Chain-of-Verification Reduces Hallucination in Large Language Models. arXiv:2309.11495.
 - [5] Gao, T., Yao, X., & Chen, D. (2021). SimCSE: Simple Contrastive Learning of Sentence Embeddings. EMNLP 2021.
 - [6] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. AISEC Workshop @ CCS 2023.
 - [7] Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. W. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. ICML 2020.
 - [8] Hong, S., et al. (2023). MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. arXiv:2308.00352.
 - [9] Izacard, G., & Grave, E. (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. EACL 2021.
 - [10] Kadavath, S., et al. (2022). Language Models (Mostly) Know What They Know. arXiv:2207.05221.
 - [11] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
 - [12] Li, J., Cheng, X., Zhao, W. X., Nie, J. Y., & Wen, J. R. (2023). HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. EMNLP 2023.
 - [13] Li, G., Hammoud, H. A., Itani, H., Khizbullin, D., & Ghanem, B. (2023). CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society. NeurIPS 2023.
 - [14] Liu, X., et al. (2023). AgentBench: Evaluating LLMs as Agents. ICLR 2024.
 - [15] Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. EMNLP 2023.
 - [16] Min, S., et al. (2023). FActScoring: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. EMNLP 2023.
 - [17] Ouyang, L., et al. (2022). Training Language Models to Follow Instructions with Human Feedback. NeurIPS 2022.
 - [18] Park, J. S., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023.
 - [19] Perez, F., & Ribeiro, I. (2022). Ignore Previous Prompt: Attack Techniques For Language Models. NeurIPS 2022 Workshop on Machine Learning Safety.
 - [20] Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. Journal of Machine Learning Research, 21, 1–67.
-

-
- [21] Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2022). MuSiQue: Multihop Questions via Single-hop Question Composition. TACL 2022.
 - [22] Wang, G., et al. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.
 - [23] Wei, A., Haghtalab, N., & Steinhardt, J. (2023). Jailbroken: How Does LLM Safety Training Fail? NeurIPS 2023.
 - [24] Wu, Q., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.
 - [25] Zhong, W., et al. (2023). MemoryBank: Enhancing Large Language Models with Long-Term Memory. AAAI 2024.
-